

At TechVision Inc., AI Engineer Sarah is tasked with optimizing the data pipeline for a new image classification project. She needs to decide whether to use bulk loading or sequential loading. How should she approach this decision, considering memory usage and training speed?

Select sequential loading for lower memory usage but potentially slower training speed.

Choose bulk loading to ensure both low memory usage and slow training speed.

Sequential loading is preferred for faster training speeds in all scenarios.

Use bulk loading to minimize memory usage and maximize training speed.

Sequential loading uses less memory as it processes data in smaller, manageable chunks, which may slow down training speed but is beneficial for memory-constrained environments.

DeepAI Solutions is developing a new deep learning model. Engineer Tom is implementing data augmentation using Keras. What is a key consideration he should keep in mind when applying these techniques?

Ensure that the augmented data does not exceed the original dataset size.

Keras automatically handles all augmentation processes without any user input.

Augmentation techniques should be relevant and enhance the model's ability to generalize.

Data augmentation should be applied after the model training is complete.

Effective augmentation techniques introduce variability that helps the model generalize better to unseen data.

At InnovateAI, data scientist Emily is using PyTorch to develop a flexible image pipeline. What is an important factor she should consider when implementing the Dataset and DataLoader classes?

The Dataset class should be used for data augmentation only.

The Dataset class is responsible for the model's training speed.

The DataLoader class automatically determines the optimal batch size.

DataLoader can be used to manage memory usage by controlling batch size and data shuffling.

Defined roles and approvals help ensure nothing critical is missed during the release process.

Why is `os.path.join(dir_non_agri, non_agri_images)` preferred over string concatenation for file paths?

It automatically downloads the file.

It ensures platform-independent path separators.

It validates that the file exists.

It converts relative paths to absolute paths.

`os.path.join` inserts the correct separator (/ or \) depending on the operating system.

Fill in the blank: The generator reshuffles indices at the start of _____.

The script

Validation only

Each epoch

Each batch

The indices are shuffled once per while-loop iteration at the start of an epoch.

Consider the following code snippet:

```
def imshow(img):  
    img = img / 2 + 0.5  
    npimg = img.numpy()  
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
```

The line *img = img / 2 + 0.5* assumes the tensor was previously normalized by dividing by 0.5 after subtracting 0.5. What original normalization call matches that assumption?

No normalization was applied

Normalize(mean=[0.5,0.5,0.5], std=[0.5,0.5,0.5])

Normalize(mean=[0.0], std=[1.0])

Normalize(mean=[0.5], std=[2.0])

Subtracting 0.5 and dividing by 0.5 maps [-1 to 1]; reversing it requires *img/2+0.5*.

datasets.ImageFolder determines class labels by which of the following?

Reading a CSV file called labels.csv

Hashing file names

The alphabetical order of immediate subfolder names

EXIF metadata inside each image

ImageFolder scans each subdirectory, sorts the names alphabetically, and assigns indices 0...N-1 of the folders to classes accordingly.